

PROGRAMMING CONCEPTS USING JAVA



WEEK 1-4

Programming languages

- A language is a medium for communication
- Programming languages communicate computational instructions
- Originally, directly connected to architecture
 - Memory locations store values, registers allow arithmetic
 - Load a value from memory location M into register R
 - Add the contents of register R_1 and R_2 and store the result back in R_1
 - Write the value in R_1 to memory location M'
- Tedious and error-prone

Abstraction

- Abstractions used in computational thinking
 - Assigning values to named variables
 - Conditional execution
 - Iteration
 - Functions / procedures, recursion
 - Aggregate data structures — arrays, lists, dictionaries
- **Express** such ideas in the programming language
 - Translate “high level” programming language to “low level” machine language
 - Compilers, interpreters
- Trade off expressiveness for efficiency
 - Less control over how code is mapped to the architecture
 - But fewer errors due to mismatch between intent and implementation

Styles of programming

- Imperative vs declarative
- Imperative
 - **How** to compute
 - Step by step instructions on what is to be done
- Declarative
 - **What** the computation should produce
 - Often exploit inductive structure, express in terms of smaller computations
 - Typically avoid using intermediate variables
 - Combination of small transformations — functional programming

Imperative vs Declarative Programming, by example

- Add values in a list
- Imperative (in Python)

```
def sumlist(l):  
    mysum = 0  
    for x in l:  
        mysum = mysum + x  
    return(mysum)
```
- Intermediate values `mysum`, `x`
- Explicit iteration to examine each element of the list
- Declarative (in Python)

```
def sumlist(l):  
    if l == []:  
        return(0)  
    else:  
        return(l[0] + sumlist(l[1:]))
```
- Describe the desired output by induction
 - Base case: Empty list has sum 0
 - Inductive step: Add first element to the sum of the rest of the list
- No intermediate variables

Imperative vs Declarative Programming, by example, ...

- Sum of squares of even numbers upto n

- Imperative (in Python)

```
def sumsquareeven(n):
    mysum = 0
    for x in range(n+1):
        if x%2 == 0:
            mysum = mysum + x*x
    return(mysum)
```

- Can code functionally in an imperative language!

- Helps identify natural units of (reusable) code

- Declarative (in Python)

```
def even(x):  
    return(x%2 == 0)
```

```
def square(x):  
    return(x*x)
```

```
def sumsquareeven(n):
    return(
        sum(map(square,
                filter(even,
                       range(n+1)))))
```

Names, types, values

- Internally, everything is stored a sequence of bits
- No difference between data and instructions, let alone numbers, characters, booleans
 - For a compiler or interpreter, our **code** is its **data**
- We impose a notion of **type** to create some discipline
 - Interpret bit strings as “high level” concepts
 - Nature and range of allowed values
 - Operations that are permitted on these values
- Strict type-checking helps catch bugs early
 - Incorrect expression evaluation — like dimension mismatch in science
 - Incorrect assignment — expression value does not match variable type

Abstract datatypes, object-oriented programming

- Collections are important
 - Arrays, lists, dictionaries
- Abstract data types
 - Structured collection with fixed interface
 - Stack is a sequence, but only allows `push` and `pop`
 - Separate implementation from interface
 - Priority queue allows `insert` and `delete-max`
 - Can implement a priority queue using sorted or unsorted lists, or using a heap
- Object-oriented programming
 - Focus on data types
 - Functions are invoked through the object rather than passing data to the functions
 - In Python, `mylist.sort()` vs `sorted(mylist)`

What this course is about

- Explore concepts in programming languages
 - Object-oriented programming
 - Exception handling, concurrency, event-driven programming, ...
- Use Java as the illustrative language
 - Imperative, object-oriented
 - Incorporates almost all features of interest
- Discuss design decisions where relevant
 - Every language makes some compromises
- Understand and appreciate why there is a zoo of programming languages out there
- ...and why new ones are still being created

The role of types

- Interpreting data stored in binary in a consistent manner
 - View sequence of bits as integers, floats, characters, ...
 - Nature and range of allowed values
 - Operations that are permitted on these values
- Naming concepts and structuring our computation
 - Especially at a higher level
 - `Point` vs `(Float,Float)`
 - Banking application: accounts of different types, customers ...
- Catching bugs early
 - Incorrect expression evaluation — like dimension mismatch in science
 - Incorrect assignment — expression value does not match variable type

Dynamic vs static typing

- Every variable we use has a type
- How is the type of a variable determined?
- Python determines the type based on the current value
 - **Dynamic typing** — names derive type from current value
 - `x = 10` — `x` is of type `int`
 - `x = 7.5` — now `x` is of type `float`
 - An uninitialized name as no type
- **Static typing** — associate a type in advance with a name
 - Need to **declare** names and their types in advance value
 - `int x, float a, ...`
 - Cannot assign an incompatible value — `x = 7.5` is no longer legal

Dynamic vs static typing

- “Isn’t it convenient that we don’t have to declare variables in advance in Python?”
- Yes, but ...
- Difficult to catch errors, such as typos

```
def factors(n):  
    factorlist = []  
    for i in range(1,n+1):  
        if n%i == 0:  
            factorlst = factorlist + [i] # Typo!  
    return(factorlist)
```

- Empty user defined objects
 - Linked list is a sequence of objects of type `Node`
 - Convenient to represent empty linked list by `None`
 - Without declaring type of `l`, Python cannot associate a type after `l = None`

Types for organizing concepts

- Even simple type “synonyms” can help clarify code
 - 2D point is a pair `(float,float)`, 3D point is triple `(float,float,float)`
 - Create new type names `point2d` and `point3d`
 - These are synonyms for `(float,float)` and `(float,float,float)`
 - Makes intent more transparent when writing, reading and maintaining code
- More elaborate types — abstract datatypes and object-oriented programming
 - Consider a banking application
 - Data and operations related to accounts, customers, deposits, withdrawals, transfers
 - Denote accounts and customers as separate types
 - Deposits, withdrawals, transfers can be applied to accounts, not customers
 - Updating personal details applies to customers, not accounts

Static analysis

- Identify errors as early as possible — saves cost, effort
- In general, compilers cannot check that a program will work correctly
 - **Halting problem** — Alan Turing
- With variable declarations, compilers can detect type errors at compile-time — **static analysis**
 - Dynamic typing would catch these errors only when the code runs
 - Executing code also slows down due to simultaneous monitoring for type correctness
- Compilers can also perform optimizations based on static analysis
 - Reorder statements to optimize reads and writes
 - Store previously computed expressions to re-use later

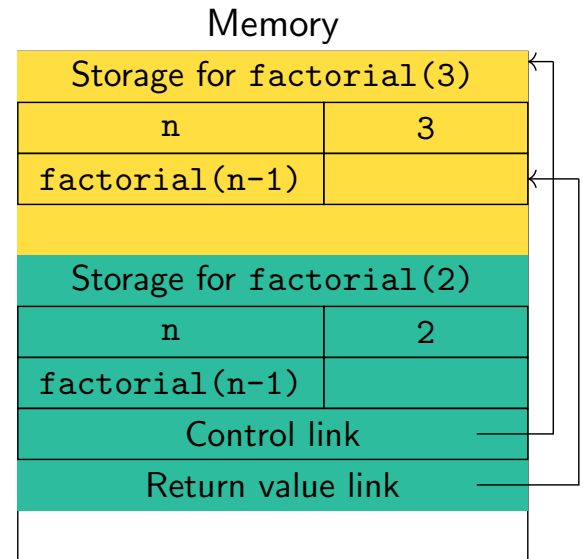
Keeping track of variables

- Variables store intermediate values during computation
 - Typically these are local to a function
 - Can also refer to global variables outside the function
 - Dynamically created data, like nodes in a list
- **Scope** of a variable
 - When the variable is available for use
 - In the following code, the `x` in `f()` is **not** in scope within call to `g()`

```
def f(l):  
    ...  
    for x in l:  
        y = y + g(x)  
    ...  
def g(m):  
    ...  
    for x in range(m):  
        ...
```
- **Lifetime** of a variable
 - How long the storage remains allocated
 - Above, lifetime of `x` in `f()` is till `f()` exits
 - “Hole in scope” — variable is alive but not in scope

Memory stack

- Each function needs storage for local variables
- Create **activation** record when function is called
- Activation records are stacked
 - Popped when function exits
 - Control link points to start of previous record
 - Return value link tells where to store result
- **Scope** of a variable
 - Variable in activation record at top of stack
 - Access global variables by following control links
- **Lifetime** of a variable
 - Storage allocated is still on the stack



- Call `factorial(3)`
- `factorial(3)` calls `factorial(2)`

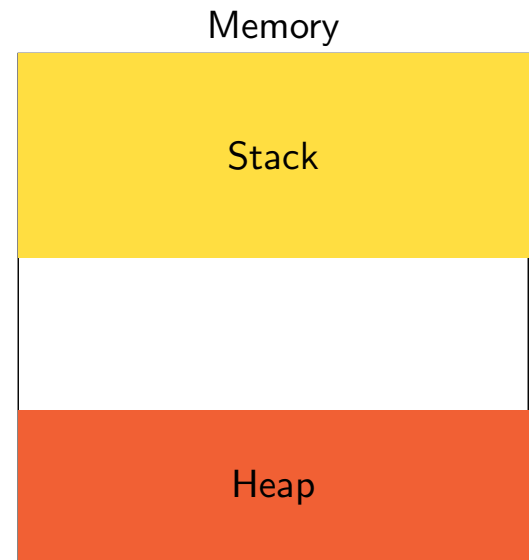
Passing arguments to functions

- When a function is called, arguments are substituted for formal parameters

```
def f(a,l):          x = 7          a = x
...                 myl = [8,9,10]  l = myl
...                 f(x,myl)        ... code for f() ...
```

- Parameters are part of the activation record of the function
 - Values are populated on function call
 - Like having implicit assignment statements at the start of the function
- Two ways to initialize the parameters
 - Call by **value** — copy the value
 - Updating the value inside the function has no **side-effect**
 - Call by **reference** — parameter points to same location as argument
 - Can have side-effects
 - Be careful: can update the contents, but cannot change the reference itself

- Function that inserts a value in a linked list
 - Storage for new node allocated inside function
 - Node should persist after function exits
 - Cannot be allocated within activation record
- Separate storage for persistent data
 - **Dynamically** allocated vs **statically** declared
 - Usually called the heap
 - Not the same as the heap data structure!
 - Conceptually, allocate heap storage from “opposite” end with respect to stack
- Heap storage outlives activation record
 - Access through some variable that is in scope



Managing heap storage

- On the stack, variables are deallocated when a function exits
- How do we “return” unused storage on the heap?
 - After deleting a node in a linked list, deleted node is now **dead** storage, unreachable
- Manual memory management
 - Programmer explicitly requests and returns heap storage
 - `p = malloc(...)` and `free(p)` in C
 - Error-prone — memory leaks, invalid assignments
- Automatic garbage collection (Java, Python, ...)
 - Run-time environment checks and cleans up dead storage — e.g., **mark-and-sweep**
 - Mark all storage that is reachable from program variables
 - Return all unmarked memory cells to free space
 - Convenience for programmer vs performance penalty

Stepwise refinement

- Begin with a high level description of the task
- Refine the task into subtasks
- Further elaborate each subtask
- Subtasks can be coded by different people
- **Program refinement** — focus on code, not much change in data structures

```
begin
  print first thousand prime numbers
end
```

```
begin
  declare table p
  fill table p with first thousand primes
  print table p
end
```

```
begin
  integer array p[1:1000]
  for k from 1 through 1000
    make p[k] equal to the kth prime number
  for k from 1 through 1000
    print p[k]
```

Data refinement

- Banking application
 - Typical functions: `CreateAccount()`, `Deposit()/Withdraw()`, `PrintStatement()`
- How do we represent each account?
 - Only need the current balance
 - Overall, an array of balances
- Refine `PrintStatement()` to include `PrintTransactions()`
 - Now we need to record transactions for each account
 - Data representation also changes
 - Cascading impact on other functions that operate on accounts

Modular software development

- Use refinement to divide the solution into **components**
- Build a **prototype** of each component to validate design
- Components are described in terms of
 - **Interfaces** — what is visible to other components, typically function calls
 - **Specification** — behaviour of the component, as visible through interface
- Improve each component independently, preserving interface and specification
- Simplest example of a component: a function
 - **Interfaces** — function header, arguments and return type
 - **Specification** — intended input-output behaviour
- Main challenge: suitable language to write specifications
 - Balance abstraction and detail, should not be another programming language!
 - Cannot algorithmically check that specification is met (halting problem!)

Programming language support for abstraction

- Control abstraction
 - Functions and procedures
 - **Encapsulate** a block of code, reuse in different contexts
- Data abstraction
 - Abstract data types (ADTs)
 - Set of values along with operations permitted on them
 - Internal representation should not be accessible
 - Interaction restricted to public interface
 - For example, when a stack is implemented as a list, we should not be able to observe or modify internal elements
- Object-oriented programming
 - Organize ADTs in a hierarchy
 - Implicit reuse of implementations — subtyping, inheritance

- An **object** is like an abstract datatype
 - Hidden data with set of public operations
 - All interaction through operations — **messages**, **methods**, **member-functions**, ...
- Uniform way of encapsulating different combinations of data and functionality
 - An object can hold single integer — e.g., a counter
 - An entire filesystem or database could be a single object
- Distinguishing features of object-oriented programming
 - Abstraction
 - Subtyping
 - Dynamic lookup
 - Inheritance

History of object-oriented programming

- Objects first introduced in **Simula** — simulation language, 1960s
- Event-based simulation follows a basic pattern
 - Maintain a queue of events to be simulated
 - Simulate the event at the head of the queue
 - Add all events it spawns to the queue
- Challenges
 - Queue must be well-typed, yet hold all types of events
 - Use a generic simulation operation across different types of events
 - Avoid elaborate checking of cases

```
Q := make-queue(first event)
repeat
  remove next event e from Q
  simulate e
  place all events generated
    by e on Q
until Q is empty
```

Abstraction

- Objects are similar to abstract datatypes
 - Public interface
 - Private implementation
 - Changing the implementation should not affect interactions with the object
- Data-centric view of programming
 - Focus on what data we need to maintain and manipulate
- Recall that stepwise refinement could affect both code and data
 - Tying methods to data makes this easier to coordinate
 - Refining data representation naturally tied to updating methods that operate on the data

Subtyping

- Recall the Simula event queue
 - A well-typed queue holds values of a fixed type
 - In practice, the queue holds different types of objects
 - How can this be reconciled?
- Arrange types in a hierarchy
 - A **subtype** is a specialization of a type
 - If **A** is a subtype of **B**, wherever an object of type **B** is needed, an object of type **A** can be used
 - Every object of type **A** is also an object of type **B**
 - Think **subset** — if $X \subseteq Y$, every $x \in X$ is also in Y
- If **f()** is a method in **B** and **A** is a subtype of **B**, every object of **A** also supports **f()**
 - Implementation of **f()** can be different in **A**

Dynamic lookup

- Whether a method can be invoked on an object is a static property — type-checking
- How the method acts is a dynamic property of how the object is implemented
 - In the simulation queue, all events support a `simulate` method
 - The action triggered by the method depends on the type of event
 - In a graphics application, different types of objects to be rendered
 - Invoke using the same operation, each object “knows” how to render itself
- Different from **overloading**
 - Operation `+` is addition for `int` and `float`
 - Internal implementation is different, but choice is determined by **static** type
- Dynamic lookup
 - A variable `v` of type `B` can refer to an object of subtype `A`
 - Static type of `v` is `B`, but method implementation depends on **run-time** type `A`

Inheritance

- Re-use of implementations
- Example: different types of employees
 - `Employee` objects store basic personal data, date of joining
 - `Manager` objects can add functionality
 - Retain basic data of `Employee` objects
 - Additional fields and functions: date of promotion, seniority (in current role)
- Usually one hierarchy of types to capture both subtyping and inheritance
 - `A` can inherit from `B` iff `A` is a subtype of `B`
- Philosophically, however the two are different
 - Subtyping is a relationship of interfaces
 - Inheritance is a relationship of implementations

Subtyping vs inheritance

- A **deque** is a double-ended queue
 - Supports `insert-front()`, `delete-front()`, `insert-rear()` and `delete-rear()`
- We can implement a stack or a queue using a deque
 - Stack: use only `insert-front()`, `delete-front()`,
 - Queue: use only `insert-rear()`, `delete-front()`,
- **Stack** and **Queue** inherit from **Deque** — reuse implementation
- But **Stack** and **Queue** are not subtypes of **Deque**
 - If `v` of type **Deque** points an object of type **Stack**, cannot invoke `insert-rear()`, `delete-rear()`
 - Similarly, no `insert-front()`, `delete-rear()` in **Queue**
- Interfaces of **Stack** and **Queue** are not compatible with **Deque**
 - In fact, **Deque** is a subtype of both **Stack** and **Queue**

Programming with objects

- Object are like abstract datatypes
 - Hidden data with set of public operations
 - All interaction through operations — **messages**, **methods**, **member-functions**, ...
- **Class**
 - Template for a data type
 - How data is stored
 - How public functions manipulate data
- **Object**
 - Concrete instance of template
 - Each object maintains a separate copy of local data
 - Invoke methods on objects — send a message to the object

Example: 2D points

- A point has coordinates (x, y)
 - Each point object stores its own internal values x and y — instance variables
 - For a point p , the local values are $p.x$ and $p.y$
 - `self` is a special name referring to the current object — `self.x`, `self.y`
- When we create an object, we need to set it up
 - Implicitly call a **constructor** function with a fixed name
 - In Python, constructor is called `__init__()`
 - Parameters are used to set up internal values
 - In Python, the first parameter is always `self`

```
class Point:
    def __init__(self, a=0, b=0):
        self.x = a
        self.y = b
```

Adding methods to a class

- Translation: shift a point by $(\Delta x, \Delta y)$
 - $(x, y) \mapsto (x + \Delta x, y + \Delta y)$
 - Update instance variables
- Distance from the origin
 - $d = \sqrt{x^2 + y^2}$
 - Does not update instance variables
 - **state** of object is unchanged

```
class Point:
    def __init__(self, a=0, b=0):
        self.x = a
        self.y = b

    def translate(self, dx, dy):
        self.x += dx
        self.y += dy

    def odistance(self):
        import math
        d = math.sqrt(self.x*self.x +
                       self.y*self.y)
        return(d)
```

Changing the internal implementation

- **Polar coordinates:** (r, θ) , not (x, y)

- $r = \sqrt{x^2 + y^2}$
- $\theta = \tan^{-1}(y/x)$

- Distance from origin is just r

```
import math
class Point:
    def __init__(self,a=0,b=0):
        self.r = math.sqrt(a*a + b*b)
        if a == 0:
            self.theta = math.pi/2
        else:
            self.theta = math.atan(b/a)

    def odistance(self):
        return(self.r)
```

Changing the internal implementation

- **Polar coordinates:** (r, θ) , not (x, y)

- $r = \sqrt{x^2 + y^2}$
- $\theta = \tan^{-1}(y/x)$

- Distance from origin is just r

- Translation

- Convert (r, θ) to (x, y)
- $x = r \cos \theta$, $y = r \sin \theta$
- Recompute r, θ from $(x + \Delta x, y + \Delta y)$

- Interface has not changed

- User need not be aware whether representation is (x, y) or (r, θ)

```
def translate(self,dx,dy):
    x = self.r*math.cos(self.theta)
    y = self.r*math.sin(self.theta)
    x += dx
    y += dy
    self.r = math.sqrt(x*x + y*y)
    if x == 0:
        self.theta = math.pi/2
    else:
        self.theta = math.atan(y/x)
```

Abstraction

- User should not know whether `Point` uses `(x,y)` or `(r,theta)`
 - Interface remains identical
 - Even constructor is the same
- Python allows direct access to instance variables from outside the class

```
p = Point(5,7)
p.x = 4 # Point is now (4,7)
```

 - Breaks the abstraction
 - Changing the internal implementation of `Point` can have impact on other code
- Rely on programmer discipline

```
class Point:
    def __init__(self,a=0,b=0):
        self.x = a
        self.y = b

class Point:
    def __init__(self,a=0,b=0):
        self.r = math.sqrt(a*a + b*b)
        if a == 0:
            self.theta = math.pi/2
        else:
            self.theta = math.atan(b/a)
```

Subtyping and inheritance

- Define `Square` to be a subtype of `Rectangle`
 - Different constructor
 - Same instance variables
- The following is legal

```
s = Square(5)
a = s.area()
p = s.perimeter()
```

 - `Square` inherits definitions of `area()` and `perimeter()` from `Rectangle`

```
class Rectangle:
    def __init__(self,w=0,h=0):
        self.width = w
        self.height = h

    def area(self):
        return(self.width*self.height)

    def perimeter(self):
        return(2*(self.width+self.height))

class Square(Rectangle):
    def __init__(self,s=0):
        self.width = s
        self.height = s
```

Subtyping and inheritance ...

- Can change the instance variable in `Square`
 - `self.side`
- The following gives a run-time error

```
s = Square(5)
a = s.area()
p = s.perimeter()
```

 - `Square` inherits definitions of `area()` and `perimeter()` from `Rectangle`
 - But `s.width` and `s.height` have not been defined!
 - Subtype is not forced to be an extension of the parent type

```
class Rectangle:
    def __init__(self,w=0,h=0):
        self.width = w
        self.height = h

    def area(self):
        return(self.width*self.height)

    def perimeter(self):
        return(2*(self.width+self.height))

class Square(Rectangle):
    def __init__(self,s=0):
        self.side = s
```

Subtyping and inheritance ...

- Subclass and parent class are usually developed separately
- Implementor of `Rectangle` changes the instance variables
- The following gives a run-time error

```
s = Square(5)
a = s.area()
p = s.perimeter()
```

 - `Square` constructor sets `s.width` and `s.height`
 - But the instance variable names have changed!
 - Why should `Square` be affected by this?

```
class Rectangle:
    def __init__(self,w=0,h=0):
        self.wd = w
        self.ht = h

    def area(self):
        return(self.wd*self.ht)

    def perimeter(self):
        return(2*(self.wd+self.ht))

class Square(Rectangle):
    def __init__(self,s=0):
        self.width = s
        self.height = s
```

Subtyping and inheritance ...

- Need a mechanism to hide private implementation details
 - **Declare** component private or public
- Working within privacy constraints
 - Instance variables **wd** and **ht** of **Rectangle** are private
 - How can the constructor for **Square** set these private variables?
 - **Square** does (and should) not know the names of the private instance variables
- Need to have elaborate declarations
 - Type and visibility of variables
- Static type checking catches errors early

```
class Rectangle:
    def __init__(self,w=0,h=0):
        self.wd = w
        self.ht = h

    def area(self):
        return(self.wd*self.ht)

    def perimeter(self):
        return(2*(self.wd+self.ht))

class Square(Rectangle):
    def __init__(self,s=0):
        self.width = s
        self.height = s
```

Getting started

The C Programming Language,
Brian W Kernighan, Dennis M Ritchie

The only way to learn a new programming language is by writing programs in it. The first program is the same for all languages.

Print the words

hello, world

This is a big hurdle; to leap over it you have to create the program text somewhere, compile it successfully, load it, run it, and find out where your output went. With these mechanical details mastered, everything else is comparatively easy

- In Python

```
print("hello, world")
```

- ...C

```
#include <stdio.h>
main()
{
    printf("hello, world\n");
}
```

- ...and Java

```
public class helloworld{
    public static void main(String[] args)
    {
        System.out.println("hello, world");
    }
}
```

Why so complicated?

- Let's unpack the syntax
- All code in Java lives within a class
 - No free floating functions, unlike Python and other languages
 - Modifier `public` specifies visibility
- How does the program start?
 - Fix a function name that will be called by default
 - From C, the convention is to call this function `main()`

```
public class helloworld{  
    public static void main(String[] args)  
    {  
        System.out.println("hello, world");  
    }  
}
```

Why so complicated ...

- Need to specify input and output types for `main()`
 - The **signature** of `main()`
 - Input parameter is an array of strings; command line arguments
 - No output, so return type is `void`
- Visibility
 - Function has to be available to run from outside the class
 - Modifier `public`

```
public class helloworld{  
    public static void main(String[] args)  
    {  
        System.out.println("hello, world");  
    }  
}
```

Why so complicated ...

■ Availability

- Functions defined inside classes are attached to objects
- How can we create an object before starting?
- Modifier `static` — function that exists independent of dynamic creation of objects

```
public class helloworld{  
    public static void main(String[] args)  
    {  
        System.out.println("hello, world");  
    }  
}
```

Why so complicated ...

■ The actual operation

- `System` is a public class
- `out` is a `stream` object defined in `System`
 - Like a file handle
 - Note that `out` must also be `static`
- `println()` is a method associated with streams
 - Prints argument with a newline, like Python `print()`

```
public class helloworld{  
    public static void main(String[] args)  
    {  
        System.out.println("hello, world");  
    }  
}
```

■ Punctuation `{`, `}`, `;` to delimit blocks, statements

- Unlike layout and indentation in Python

Compiling and running Java code

- A Java program is a collection of classes
- Each class is defined in a separate file with the same name, with extension `java`

```
public class helloworld{  
    public static void main(String[] args)  
    {  
        System.out.println("hello, world");  
    }  
}
```

- Class `helloworld` in `helloworld.java`
- Java programs are usually interpreted on **Java Virtual Machine (JVM)**
 - JVM provides a uniform execution environment across operating systems
 - Semantics of Java is defined in terms of JVM, OS-independent
 - **"Write once, run anywhere"**

Compiling and running Java code

- `javac` compiles into JVM **bytecode**
 - `javac helloworld.java` creates bytecode file `helloworld.class`
- `java helloworld` interprets and runs bytecode in `helloworld.class`
- **Note:**

```
public class helloworld{  
    public static void main(String[] args)  
    {  
        System.out.println("hello, world");  
    }  
}
```

- `javac` requires file extension `.java`
- `java` should not be provided file extension `.class`
- `javac` automatically follows dependencies and compiles all classes required
 - Sufficient to trigger compilation for class containing `main()`

Scalar types

- In an object-oriented language, all data should be encapsulated as objects
- However, this is cumbersome
 - Useful to manipulate numeric values like conventional languages
- Java has eight primitive scalar types
 - `int`, `long`, `short`, `byte`
 - `float`, `double`
 - `char`
 - `boolean`
- Size of each type is fixed by JVM
 - Does not depend on native architecture

Type	Size in bytes
<code>int</code>	4
<code>long</code>	8
<code>short</code>	2
<code>byte</code>	1
<code>float</code>	4
<code>double</code>	8
<code>char</code>	2
<code>boolean</code>	1

- 2-byte `char` for Unicode

Declarations, assigning values

- We **declare** variables before we use them

```
int x, y;
double y;
char c;
boolean b1, b2;
```

 - Note the semicolons after each statement
- The assignment statement works as usual

```
int x,y;
x = 5;
y = 7;
```
- Characters are written with single-quotes (only)

```
char c,d;

c = 'x';
d = '\u03C0'; // Greek pi, unicode
```

 - Double quotes denote **strings**
- Boolean constants are `true`, `false`

```
boolean b1, b2;

b1 = false;
b2 = true;
```

Initialization, constants

- Declarations can come anywhere

```
int x;  
x = 10;  
double y;
```

- Use this judiciously to retain readability

- Initialize at time of declaration

```
int x = 10;  
double y = 5.7;
```

- Can we declare a value to be a constant?

```
float pi = 3.1415927f;
```

```
pi = 22/7; // Disallow?
```

- Note: Append `f` after number for `float`, else interpreted as `double`

- Modifier `final` indicates a constant

```
final float pi = 3.1415927f;
```

```
pi = 22/7; // Flagged as error;
```

Operators, shortcuts, type casting

- Arithmetic operators are the usual ones

- +, -, *, /, %

- No separate integer division operator //

- When both arguments are integer, `/` is integer division

```
float f = 22/7;    // Value is 3.0
```

- Note implicit conversion from `int` to `float`

- No exponentiation operator, use `Math.pow()`

- `Math.pow(a,n)` returns a^n

- Special operators for incrementing and decrementing integers

```
int a = 0, b = 10;
a++;    // Same as a = a+1
b--;    // Same as b = b-1
```

- Shortcut for updating a variable

```
int a = 0, b = 10;
a += 7;    // Same as a = a+7
b *= 12;   // Same as b = b*12
```

Strings

- `String` is a built in class

```
String s,t;
```

- String constants enclosed in double quotes

```
String s = "Hello", t = "world";
```

- `+` is overloaded for string concatenation

```
String s = "Hello";
String t = "world";
String u = s + " " + t;
// "Hello world"
```

- Strings are **not** arrays of characters

- Cannot write

```
s[3] = 'p';  
s[4] = '!';
```

- Instead, invoke method `substring` in class `String`

- ```
■ s = s.substring(0,3) + "p!";
```

- If we change a `String`, we get a new object

- After the update, `s` points to a new `String`

- Java does automatic garbage collection

# Arrays

- Arrays are also objects

- Typical declaration

```
int[] a;
a = new int[100];
```

- Or `int a[]` instead of `int[] a`
- Combine as `int[] a = new int[100];`

- `a.length` gives size of `a`

- Note, for `String`, it is a method `s.length()`!

- Array indices run from 0 to `a.length-1`

- Size of the array can vary

- Array constants:  $\{v1, v2, v3\}$

- For example

```
int[] a;
int n;
```

```
n = 10;
a = new int[n];
```

```
n = 20;
a = new int[n];
```

$$a = \{2, 3, 5, 7, 11\};$$

# Control flow

- Program layout
  - Statements end with semi-colon
  - Blocks of statements delimited by braces
- Conditional execution
  - `if (condition) { ... } else { ... }`
- Conditional loops
  - `while (condition) { ... }`
  - `do { ... } while (condition)`
- Iteration
  - Two kinds of `for`
- Multiway branching – `switch`

## Conditional execution

- `if (c) {...} else {...}`
  - `else` is optional
  - Condition must be in parentheses
  - If body is a single statement, braces are not needed
- No `elif`, à la Python
  - Indentation is not forced
  - Just align `else if`
  - Nested `if` is a single statement, no separate braces required
- No surprises
- Aside: no `def` for function definition

```
public class MyClass {

 ...

 public static int sign(int v) {
 if (v < 0) {
 return(-1);
 } else if (v > 0) {
 return(1);
 } else {
 return(0);
 }
 }
}
```

# Conditional loops

## ■ while (c) {...}

- Condition must be in parentheses
- If body is a single statement, braces are not needed

```
public class MyClass {
 ...

 public static int sumupto(int n) {
 int sum = 0;

 while (n > 0){
 sum += n;
 n--;
 }

 return(sum);
 }
}
```

# Conditional loops

## ■ while (c) {...}

- Condition must be in parentheses
- If body is a single statement, braces are not needed

## ■ do {...} while (c)

- Condition is checked at the end of the loop
- At least one iteration
- Useful for interactive user input

```
do {
 read input;
} while (input-condition);
```

```
public class MyClass {
 ...

 public static int sumupto(int n) {
 int sum = 0;
 int i = 0;

 do {
 sum += i;
 i++;
 } while (i <= n);

 return(sum);
 }
}
```

# Iteration

- `for` loop is inherited from C
- `for (init; cond; upd) {...}`
  - `init` is initialization
  - `cond` is terminating condition
  - `upd` is update
- Intended use is  
`for(i = 0; i < n; i++){...}`
- Completely equivalent to  

```
i = 0;
while (i < n) {
 i++;
}
```

```
public class MyClass {
 ...

 public static int sumarray(int[] a) {
 int sum = 0;
 int n = a.length;
 int i;

 for (i = 0; i < n; i++){
 sum += a[i];
 }

 return(sum);
 }
}
```

# Iteration

- Intended use is  
`for(i = 0; i < n; i++){...}`
- Completely equivalent to  

```
i = 0;
while (i < n) {
 i++;
}
```
- However, not good style to write `for` instead of `while`
- Can define loop variable within loop
  - The scope of `i` is local to the loop
  - An instance of more general local scoping allowed in Java

```
public class MyClass {
 ...

 public static int sumarray(int[] a) {
 int sum = 0;
 int n = a.length;

 for (int i = 0; i < n; i++){
 sum += a[i];
 }

 return(sum);
 }
}
```

# Iterating over elements directly

- Java later introduced a `for` in the style of Python

```
for x in l:
 do something with x
```

- Again `for`, different syntax

```
for (type x : a)
 do something with x;
}
```

- It appears that loop variable **must** be declared in local scope for this version of `for`

```
public class MyClass {
 ...

 public static int sumarray(int[] a) {
 int sum = 0;
 int n = a.length;

 for (int v : a){
 sum += v;
 }

 return(sum);
 }
}
```

# Multiway branching

- `switch` selects between different options
- Be careful, default is to “fall through” from one case to the next
  - Need to explicitly `break` out of switch
  - `break` available for loops as well
  - Check the Java documentation
- Options have to be constants
  - Cannot use conditional expressions
- Aside: here return type is `void`
  - Non-`void` return type requires an appropriate `return` value

```
public static void printsign(int v) {
 switch (v) {
 case -1: {
 System.out.println("Negative");
 break;
 }
 case 1: {
 System.out.println("Positive");
 break;
 }
 case 0: {
 System.out.println("Zero");
 break;
 }
 }
}
```

# Classes and objects

- A **class** is a template for an encapsulated type
- An **object** is an instance of a class
- How do we create objects?
- How are objects initialized?

## Defining a class

- Definition block using **class**, with class name
  - Modifier **public** to indicate visibility
  - Java allows **public** to be omitted
  - Default visibility is public to **package**
  - Packages are administrative units of code
  - All classes defined in same directory form part of same package
- Instance variables
  - Each concrete object of type **Date** will have local copies of **date**, **month**, **year**
  - These are marked **private**
  - Can also have **public** instance variables, but breaks encapsulation

```
public class Date {

 private int day, month, year;

 ...
}
```

# Creating objects

- Declare type using class name
- `new` creates a new object
  - How do we set the instance variables?
- Can add methods to update values
  - `this` is a reference to current object
  - Can omit `this` if reference is unambiguous

```
public void UseDate() {
 Date d;
 d = new Date();
 ...
}

public class Date {
 private int day, month, year;

 public void setDate(int d, int m,
 int y){

 day = d;
 month = m;
 year = y;
 }
}
```

# Creating objects

- Declare type using class name
- `new` creates a new object
  - How do we set the instance variables?
- Can add methods to update values
  - `this` is a reference to current object
  - Can omit `this` if reference is unambiguous
- What if we want to check the values?
  - Methods to read and report values
- **Accessor** and **Mutator** methods

```
public class Date {
 ...

 public int getDay(){
 return(day);
 }

 public int getMonth(){
 return(month);
 }

 public int getYear(){
 return(year);
 }
}
```

# Initializing objects

- Would be good to set up an object when we create it
  - Combine `new Date()` and `setDate()`
- **Constructors** — special functions called when an object is created
  - Function with the same name as the class
  - `d = new Date(13,8,2015);`
- Constructors with different signatures
  - `d = new Date(13,8);` sets `year` to 2021
  - Java allows function overloading — same name, different signatures
    - Python: default (optional) arguments, no overloading

```
public class Date {
 private int day, month, year;

 public Date(int d, int m, int y){
 day = d;
 month = m;
 year = y;
 }

 public Date(int d, int m){
 day = d;
 month = m;
 year = 2021;
 }
}
```

## Constructors ...

- A later constructor can call an earlier one using `this`
- If no constructor is defined, Java provides a default constructor with empty arguments
  - `new Date()` would implicitly invoke this
  - Sets instance variables to sensible defaults
  - For instance, `int` variables set to 0
  - Only valid if *no* constructor is defined
  - Otherwise need an explicit constructor without arguments

```
public class Date {
 private int day, month, year;

 public Date(int d, int m, int y){
 day = d;
 month = m;
 year = y;
 }

 public Date(int d, int m){
 this(d,m,2021);
 }
}
```

# Copy constructors

- Create a new object from an existing one
- Copy constructor takes an object of the same type as argument
  - Copies the instance variables
  - Use object name to disambiguate which instance variables we are talking about
  - Note that private instance variables of argument are visible
- Shallow copy vs deep copy
  - Want new object to be disjoint from old one
  - If instance variable are objects, we may end up aliasing rather than copying
  - Discuss later — cloning objects

```
public class Date {
 private int day, month, year;

 public Date(Date d){
 this.day = d.day;
 this.month = d.month;
 this.year = d.year;
 }
}
```

```
public void UseDate() {
 Date d1,d2;
 d1 = new Date(12,4,1954);
 d2 = new Date(d1);
}
```

# Interacting with a Java program

- We have seen how to print data
  - `System.out.println("hello, world");`
- How do we read data

# Reading input

- Simplest to use is the `Console` class
  - Functionality similar to Python `input()`
- Defined within `System`
  - Two methods, `readLine` and `readPassword`
  - `readPassword` does not echo characters on the screen
  - `readLine` returns a string (like Python `input()`)
  - `readPassword` returns an array of `char` — for security reasons

```
Console cons = System.console();
String username =
 cons.readLine("User name: ");
char[] passwd =
 cons.readPassword("Password: ");
```

- More general `Scanner` class
  - Allows more granular reading of input
  - Read a full line, or read an integer, ...

```
Scanner in = new Scanner(System.in);
String name = in.nextLine();
int age = in.nextInt();
....
```

# Generating output

- `System.out.println(arg)` prints `arg` and goes to a new line
  - Implicitly converts argument to a string
- `System.out.print(arg)` is similar, but does not advance to a new line
- `System.out.printf(arg)` generates formatted output
  - Same conventions as `printf` in C
  - Read the documentation

# Algorithms + Data Structures = Programs

- Title of Niklaus Wirth's introduction to Pascal
- Traditionally, algorithms come first
- Structured programming
  - Design a set of procedures for specific tasks
  - Combine them to build complex systems
- Data representation comes later
  - Design data structures to suit procedural manipulations

## Object Oriented design

- Reverse the focus
- First identify the data we want to maintain and manipulate
- Then identify algorithms to operate on the data
- Claim: works better for large systems
- Example: simple web browser
  - 2000 procedures manipulating global data
  - ... vs 100 classes, each with about 20 methods
  - Much easier to grasp the design
  - Debugging: an object is in an incorrect state
  - Search among 20 methods rather than 2000 procedures

# Object Oriented design: Example

- An order processing system typically involves
  - Items
  - Orders
  - Shipping addresses
  - Payments
  - Accounts
- What happens to these objects?
  - Items are **added** to orders
  - Orders are **shipped**, **cancelled**
  - Payments are **accepted**, **rejected**
- **Nouns** signify objects, **verbs** denote methods that operate on objects
  - Associate with each order, a method to add an item

## Designing objects

- Behaviour — what methods do we need to operate on objects?
- State — how does the object react when methods are invoked?
  - **State** is the information in the instance variables
  - Encapsulation — should not change unless a method operates on it
- Identity — distinguish between different objects of the same class
  - State may be the same — two orders may contain the same item
- These features interact
  - State will typically affect behaviour
  - Cannot add an item to an order that has been shipped
  - Cannot ship an empty order

# Relationship between classes

- Dependence

- **Order** needs **Account** to check credit status
- **Item** does not depend on **Account**
- Robust design minimizes dependencies, or **coupling** between classes

- Aggregation

- `Order` contains `Item` objects

## ■ Inheritance

- One object is a specialized versions of another
- `ExpressOrder` inherits from `Order`
- Extra methods to compute shipping charges, priority handling

# A Java class

- An `Employee` class
- Two private instance variables
- Some constructors to set up the object
- Accessor and mutator methods to set instance variables
- A public method to compute bonus

```
public class Employee{
 private String name;
 private double salary;

 // Some Constructors ...

 // "mutator" methods
 public boolean setName(String s){ ... }
 public boolean setSalary(double x){ ... }

 // "accessor" methods
 public String getName(){ ... }
 public double getSalary(){ ... }

 // other methods
 public double bonus(float percent){
 return (percent/100.0)*salary;
 }
}
```

# Subclasses

- Managers are special types of employees with extra features

```
public class Manager extends Employee{
 private String secretary;
 public boolean setSecretary(name s){ ... }
 public String getSecretary(){ ... }
}
```

- **Manager** objects inherit other fields and methods from **Employee**
  - Every **Manager** has a **name**, **salary** and methods to access and manipulate these.
- **Manager** is a **subclass** of **Employee**
  - Think of subset

# Subclasses

- **Manager** objects do not automatically have access to private data of parent class.
  - Common to extend a parent class written by someone else
- How can a constructor for **Manager** set instance variables that are private to **Employee**?
- Some constructors for **Employee**
- Use parent class's constructor using **super**
- A constructor for **Manager**

```
public class Employee{
 ...
 public Employee(String n, double s){
 name = n; salary = s;
 }
 public Employee(String n){
 this(n,500.00);
 }
}

public class Manager extends Employee{
 ..
 public Manager(String n, double s, String sn){
 super(n,s); /* super calls
 Employee constructor */
 secretary = sn;
 }
}
```

# Inheritance

- In general, subclass has more features than parent class
  - Subclass **inherits** instance variables, methods from parent class
- Every **Manager** is an **Employee**, but not vice versa!
- Can use a subclass in place of a superclass

```
Employee e = new Manager(...)
```
- But the following will not work

```
Manager m = new Employee(...)
```
- Recall
  - `int[] a = new int[100];`
  - Why the seemingly redundant reference to `int` in `new`?
- One can now presumably write

```
Employee[] e = new Manager(...)[100]
```

## Subclasses and inheritance

- A subclass extends a parent class
- Subclass inherits instance variables and methods from the parent class
- Subclasses cannot see private components of parent class
- Subclass can add more instance variables and methods
- Can also override methods

```
public class Employee{
 private String name;
 private double salary;

 public boolean setName(String s){ ... }
 public boolean setSalary(double x){ ... }
 public String getName(){ ... }
 public double getSalary(){ ... }

 public double bonus(float percent){
 return (percent/100.0)*salary;
 }
}

public class Manager extends Employee{
 private String secretary;
 public boolean setSecretary(name s){ ... }
 public String getSecretary(){ ... }
}
```

# Dynamic dispatch

- `Manager` can redefine `bonus()`

```
double bonus(float percent){
 return 1.5*super.bonus(percent);
}
```

  - Uses parent class `bonus()` via `super`
  - **Overrides** definition in parent class
- Consider the following assignment

```
Employee e = new Manager(...)
```
- Can we invoke `e.setSecretary()`?
  - `e` is declared to be an `Employee`
  - Static typechecking — `e` can only refer to methods in `Employee`
- What about `e.bonus(p)`? Which `bonus()` do we use?
  - **Static**: Use `Employee.bonus()`
  - **Dynamic**: Use `Manager.bonus()`
- **Dynamic dispatch** (dynamic binding, late method binding, ...) turns out to be more useful
  - Default in Java, optional in languages like C++ (**virtual** function)

# Polymorphism

- Every `Employee` in `emparray` “knows” how to calculate its `bonus` correctly!
- Recall the event simulation loop that motivated Simula to introduce objects

```
Q := make-queue(first event)
repeat
 remove next event e from Q
 simulate e
 place all events generated
 by e on Q
until Q is empty
```

# Polymorphism

- Every `Employee` in `emparray` “knows” how to calculate its `bonus` correctly!
- Recall the event simulation loop that motivated Simula to introduce objects
- Also referred to as **runtime polymorphism** or **inheritance polymorphism**

```
Employee[] emparray = new Employee[2];
Employee e = new Employee(...);
Manager m = new Manager(...);

emparray[0] = e;
emparray[1] = m;

for (i = 0; i < emparray.length; i++){
 System.out.println(emparray[i].bonus(5.0));
}
```

## Functions, signatures and overloading

- Signature of a function is its name and the list of argument types
- Can have different functions with the same name and different signatures
  - For example, multiple constructors
- Java class `Arrays` has a method `sort` to sort arbitrary scalar arrays
- Made possible by overloaded methods defined in class `Arrays`

```
double[] darr = new double[100];
int[] iarr = new int[500];
...
Arrays.sort(darr);
 // sorts contents of darr
Arrays.sort(iarr);
 // sorts contents of iarr

class Arrays{
 ...
 public static void sort(double[] a){..}
 // sorts arrays of double[]
 public static void sort(int[] a){..}
 // sorts arrays of int[]
 ...
}
```

# Functions, signatures and overloading

- **Overloading**: multiple methods, different signatures, choice is static
- **Overriding**: multiple methods, same signature, choice is static
  - `Employee.bonus()`
  - `Manager.bonus()`
- **Dynamic dispatch**: multiple methods, same signature, choice made at run-time

```
double[] darr = new double[100];
int[] iarr = new int[500];
...
Arrays.sort(darr);
 // sorts contents of darr
Arrays.sort(iarr);
 // sorts contents of iarr

class Arrays{
 ...
 public static void sort(double[] a){..}
 // sorts arrays of double[]
 public static void sort(int[] a){..}
 // sorts arrays of int[]
 ...
}
```

## Type casting

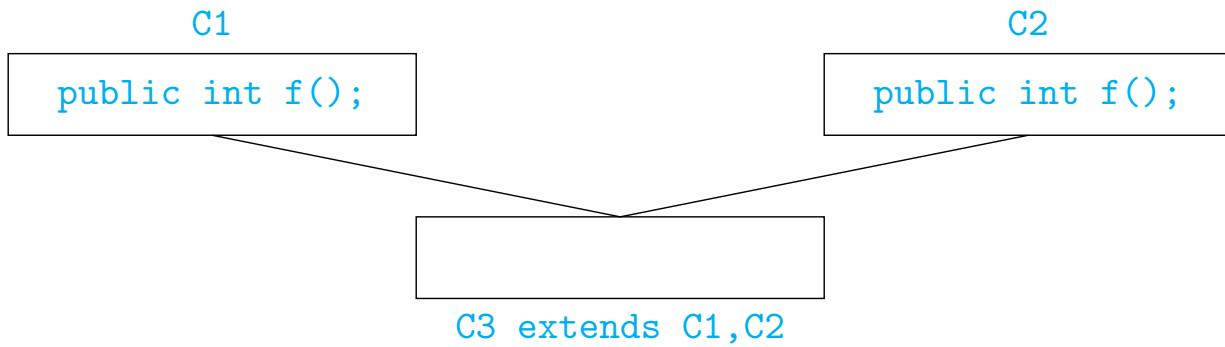
- Consider the following assignment  
`Employee e = new Manager(...)`
- Can we get `e.setSecretary()` to work?
  - Static type-checking disallows this
- Type casting — convert `e` to `Manager`  
`((Manager) e).setSecretary(s)`
- Cast fails (error at run time) if `e` is not a `Manager`

- Can test if `e` is a `Manager`  

```
if (e instanceof Manager){
 ((Manager) e).setSecretary(s);
}
```
- A simple example of **reflection** in Java
  - “Think about oneself”
- Can also use type casting for basic types  

```
double d = 29.98;
int nd = (int) d;
```

# Multiple inheritance



- Can a subclass extend multiple parent classes?
- If `f()` is not overridden, which `f()` do we use in `C3`?
- Java does not allow multiple inheritance
- C++ allows this if `C1` and `C2` have no conflict

## Java class hierarchy

- No multiple inheritance — tree-like
- In fact, there is a universal superclass `Object`
- Useful methods defined in `Object`

```
public boolean equals(Object o) // defaults to pointer equality

public String toString() // converts the values of the
 // instance variables to String
```

- For Java objects `x` and `y`, `x == y` invokes `x.equals(y)`
- To print `o`, use `System.out.println(o+"");`
  - Implicitly invokes `o.toString()`

# Java class hierarchy

- Can exploit the tree structure to write generic functions

- Example: search for an element in an array

```
public int find (Object[] objarr, Object o){
 int i;
 for (i = 0; i < objarr.length(); i++){
 if (objarr[i] == o) {return i};
 }
 return (-1);
}
```

- Recall that `==` is pointer equality, by default
- If a class overrides `equals()`, dynamic dispatch will use the redefined function instead of `Object.equals()` for `objarr[i] == o`

## Overriding functions

- For instance, a class `Date` with instance variables `day`, `month` and `year`

- May wish to override `equals()` to compare the object state, as follows

```
public boolean equals(Date d){
 return ((this.day == d.day) &&
 (this.month == d.month) &&
 (this.year == d.year));
}
```

- Unfortunately,  
`boolean equals(Date d)`  
does not override  
`boolean equals(Object o)`!

- Should write, instead

```
public boolean equals(Object d){
 if (d instanceof Date){
 Date myd = (Date) d;
 return ((this.day == myd.day) &&
 (this.month == myd.month)
 (this.year == myd.year));
 }
 return(false);
}
```

- Note the run-time type check and the cast

# Overriding functions

- Overriding looks for “closest” match
- Suppose we have `public boolean equals(Employee e)` but no `equals()` in `Manager`
- Consider

```
Manager m1 = new Manager(...);
Manager m2 = new Manager(...);
...
if (m1.equals(m2)){ ... }
```
- `public boolean equals(Manager m)` is compatible with both `boolean equals(Employee e)` and `boolean equals(Object o)`
- Use `boolean equals(Employee e)`

# Subclasses, subtyping and inheritance

- Class hierarchy provides both **subtyping** and **inheritance**
- **Subtyping**
  - Capabilities of the subtype are a superset of the main type
  - If `B` is a subtype of `A`, wherever we require an object of type `A`, we can use an object of type `B`
  - `Employee e = new Manager(...);` is legal
- **Inheritance**
  - Subtype can reuse code of the main type
  - `B` inherits from `A` if some functions for `B` are written in terms of functions of `A`
  - `Manager.bonus()` uses `Employee.bonus()`

# Subtyping vs inheritance

- Recall the following example
  - `queue`, with methods `insert-rear`, `delete-front`
  - `stack`, with methods `insert-front`, `delete-front`
  - `deque`, with methods `insert-front`, `delete-front`, `insert-rear`, `delete-rear`
- What are the subtype and inheritance relationships between these classes?
- **Subtyping**
  - `deque` has more functionality than `queue` or `stack`
  - `deque` is a subtype of both these types
- **Inheritance**
  - Can suppress two functions in a `deque` and use it as a `queue` or `stack`
  - Both `queue` and `stack` inherit from `deque`

# Subclasses, subtyping and inheritance

- Class hierarchy represents both **subtyping** and **inheritance**
- **Subtyping**
  - Compatibility of interfaces.
  - `B` is a subtype of `A` if every function that can be invoked on an object of type `A` can also be invoked on an object of type `B`.
- **Inheritance**
  - Reuse of implementations.
  - `B` inherits from `A` if some functions for `B` are written in terms of functions of `A`.
- Using one idea (hierarchy of classes) to implement both concepts blurs the distinction between the two

# Modifiers in Java

- Java uses many modifiers in declarations, to cover different features of object-oriented programming
- `public` vs `private` to support encapsulation of data
- `static`, for entities defined inside classes that exist without creating objects of the class
- `final`, for values that cannot be changed
- These modifiers can be applied to classes, instance variables and methods
- Let's look at some examples of situations where different combinations make sense

## public vs private

- Faithful implementation of encapsulation necessitates modifiers `public` and `private`
  - Typically, instance variables are `private`
  - Methods to query (accessor) and update (mutator) the state are `public`
- Can `private` methods make sense?
- Example: a `Stack` class
  - Data stored in a private array
  - Public methods to push, pop, query if empty

```
public class Stack {
 private int[] values; // array of values
 private int tos; // top of stack
 private int size; // values.length

 /* Constructors to set up values array */

 public void push (int i){

 }

 public int pop (){
 ...
 }

 public boolean is_empty (){
 return (tos == 0);
 }
}
```

## private methods

- Example: a `Stack` class
  - Data stored in a private array
  - Public methods to push, pop, query if empty
- `push()` needs to check if stack has space

```
public class Stack {
 ...
 public void push (int i){
 if (tos < size){
 values[tos] = i;
 tos = tos+1;
 }else{
 // Deal with stack overflow
 }
 ...
 }
 ...
}
```

## private methods

- Example: a `Stack` class
  - Data stored in a private array
  - Public methods to push, pop, query if empty
- `push()` needs to check if stack has space
- Deal gracefully with stack overflow
  - `private` methods invoked from within `push()` to check if stack is full and expand storage

```
public class Stack {
 ...
 public void push (int i){
 if (stack_full()){
 extend_stack();
 }
 ... // Usual push operations
 }
 ...
 private boolean stack_full(){
 return(tos == size);
 }
 private void extend_stack(){
 /* Allocate additional space,
 reset size etc */
 }
}
```

# Accessor and mutator methods

- Public methods to query and update private instance variables
- `Date` class
  - Private instance variables `day`, `month`, `year`
  - One public accessor/mutator method per instance variable
- Inconsistent updates are now possible
  - Separately set invalid combinations of `day` and `month`

```
public class Date {
 private int day, month, year;

 public void getDay(int d) {...}
 public void getMonth(int m) {...}
 public void getYear(int y) {...}

 public void setDay(int d) {...}
 public void setMonth(int m) {...}
 public void setYear(int y) {...}
}
```

# Accessor and mutator methods

- Public methods to query and update private instance variables
- `Date` class
  - Private instance variables `day`, `month`, `year`
  - One public accessor/mutator method per instance variable
- Inconsistent updates are now possible
  - Separately set invalid combinations of `day` and `month`
- Instead, allow only combined update

```
public class Date {
 private int day, month, year;

 public void getDay(int d) {...}
 public void getMonth(int m) {...}
 public void getYear(int y) {...}

 public void setDate(int d, int m, int y) {
 ...
 // Validate d-m-y combination
 }
}
```

## static components

- Use `static` for components that exist without creating objects
  - Library functions, `main()`, ...
  - Useful constants like `Math.PI`, `Integer.MAX_VALUE`
- These `static` components are also `public`
- Do `private static` components make sense?
- Internal constants for bookkeeping
  - Constructor sets unique id for each order

```
public class Order {
 private static int lastorderid = 0;

 private int orderid;

 public Order(...) {
 lastorderid++;
 orderid = lastorderid;
 ...
 }
}
```

- `lastorderid` is private static field
- Common to all objects in the class
- Be careful about concurrent updates!

## final components

- `final` denotes that a value cannot be updated
- Usually used for constants (`public` and `static` instance variables)
  - `Math.PI`, `Integer.MAX_VALUE`
- What would `final` mean for a method?
  - Cannot redefine functions at run-time, unlike Python!
- Recall **overriding**
  - Subclass redefines a method available with the same signature in the parent class
- A `final` method cannot be overridden

# Grouping together classes

- Sometimes we collect together classes under a common heading
- Classes `Circle`, `Square` and `Rectangle` are all shapes
- Create a class `Shape` so that `Circle`, `Square` and `Rectangle` extend `Shape`
- We want to force every `Shape` to define a function  
`public double perimeter()`
- Could define a function in `Shape` that returns an absurd value  
`public double perimeter() { return(-1.0); }`
- Rely on the subclass to redefine this function
- What if this doesn't happen?
  - Should not depend on programmer discipline

## Abstract classes

- A better solution
  - Provide an **abstract definition** in `Shape`  
`public abstract double perimeter();`
- Forces subclasses to provide a concrete implementation
- Cannot create objects from a class that has abstract functions
- `Shape` must itself be declared to be **abstract**

```
public abstract class Shape{
 ...
 public abstract double perimeter();
 ...
}
```

## Abstract classes ...

- Can still declare variables whose type is an abstract class

```
Shape shapearr[] = new Shape[3];
int sizearr[] = new int[3];

shapearr[0] = new Circle(...);
shapearr[1] = new Square(...);
shapearr[2] = new Rectangle(...);

for (i = 0; i < 2; i++){
 sizearr[i] = shapearr[i].perimeter();
 // each shapearr[i] calls the appropriate method
 ...
}
```

## Generic functions

- Use abstract classes to specify generic properties

```
public abstract class Comparable{
 public abstract int cmp(Comparable s);
 // return -1 if this < s,
 // 0 if this == s,
 // +1 if this > s
}
```

- Now we can sort any array of objects that extend `Comparable`

```
public class SortFunctions{
 public static void quicksort(Comparable[] a){
 ...
 // Usual code for quicksort, except that
 // to compare a[i] and a[j] we use a[i].cmp(a[j])
 }
}
```

## Generic functions ...

```
public class SortFunctions{
 public static void quicksort(Comparable[] a){
 ...
 }
}
```

- To use this definition of `quicksort`, we write

```
public class Myclass extends Comparable{
 private double size; // quantity used for comparison

 public int cmp(Comparable s){
 if (s instanceof Myclass){
 // compare this.size and ((Myclass) s).size
 // Note the cast to access s.size
 }
 }
}
```

Navigation icons: back, forward, search, etc.

## Multiple inheritance

- Can we sort `Circle` objects using the generic functions in `SortFunctions`?
  - `Circle` already extends `Shape`
  - Java does not allow `Circle` to also extend `Comparable`!

- An **interface** is an abstract class with no concrete components

```
public interface Comparable{
 public abstract int cmp(Comparable s);
}
```

- A class that extends an interface is said to **implement** it:

```
public class Circle extends Shape implements Comparable{
 public double perimeter(){...}
 public int cmp(Comparable s){...}
 ...
}
```

- Can extend only one class, but can implement multiple interfaces

Navigation icons: back, forward, search, etc.

- An interface is a purely abstract class
  - All methods are abstract
- A class **implements** an interface
  - Provide concrete code for each abstract function
- Classes can implement multiple interfaces
  - Abstract functions, so no contradictory inheritance
- Interfaces describe relevant aspects of a class
  - Abstract functions describe a specific “slice” of capabilities
  - Another class only needs to know about these capabilities

## Exposing limited capabilities

- Generic **quicksort** for any datatype that supports comparisons
- Express this capability by making the argument type **Comparable[]**
  - **Only** information that **quicksort** needs about the underlying type
  - All other aspects are irrelevant
- Describe the relevant functions supported by **Comparable** objects through an interface
- However, we **cannot** express the intended behaviour of **cmp** explicitly

```
public class SortFunctions{
 public static void quicksort(Comparable[] a){
 ...
 // Usual code for quicksort, except that
 // to compare a[i] and a[j] we use
 // a[i].cmp(a[j])
 }
}
```

```
public interface Comparable{
 public abstract int cmp(Comparable s);
 // return -1 if this < s,
 // 0 if this == s,
 // +1 if this > s
}
```

# Adding methods to interfaces

- Java interfaces extended to allow functions to be added
- Static functions
  - Cannot access instance variables
  - Invoke directly or using interface name: `Comparable.cmpdoc()`
- Default functions
  - Provide a default implementation for some functions
  - Class can override these
  - Invoke like normal method, using object name: `a[i].cmp(a[j])`

```
public interface Comparable{
 public static String cmpdoc(){
 String s;
 s = "Return -1 if this < s, ";
 s = s + "0 if this == s, ";
 s = s + "+1 if this > s.";
 return(s);
 }
}
```

```
public interface Comparable{
 public default int cmp(Comparable s) {
 return(0);
 }
}
```

# Dealing with conflicts

- Old problem of multiple inheritance returns
  - Conflict between static/default methods
- Subclass **must** provide a fresh implementation

```
public interface Person{
 public default String getName() {
 return("No name");
 }
}
```

```
public interface Designation{
 public default String getName() {
 return("No designation");
 }
}
```

```
public class Employee
 implements Person, Designation {
 ...

 public String getName(){
 ...
 }
}
```

# Dealing with conflicts

- Old problem of multiple inheritance returns
  - Conflict between static/default methods
- Subclass **must** provide a fresh implementation
- Conflict could be between a class and an interface
  - `Employee` inherits from class `Person` and implements `Designation`
  - Method inherited from the class “wins”
  - Motivated by reverse compatibility

```
public class Person{
 public String getName() {
 return("No name");
 }
}

public interface Designation{
 public default String getName() {
 return("No designation");
 }
}

public class Employee
 extends Person implements Designation {
 ...
}
```

# Nested objects

- An instance variable can be a user defined type
  - `Employee` uses `Date`
- `Date` is a public class, also available to other classes
- When could a private class make sense?

```
public class Employee{
 private String name;
 private double salary;
 private Date joindate;

 ...
}

public class Date {
 private int day, month year;

 ...
}
```

# Nested objects

- `LinkedList` is built using `Node`
- Why should `Node` be public?
  - May want to enhance with `prev` field, doubly linked list
  - Does not affect interface of `LinkedList`

```
public class Node {
 public Object data;
 public Node next;
 ...
}

public class LinkedList{
 private int size;
 private Node first;

 public Object head(){
 Object returnval = null;
 if (first != null){
 returnval = first.data;
 first = first.next;
 }
 return(returnval);
 }
}
```

# Nested objects

- `LinkedList` is built using `Node`
- Why should `Node` be public?
  - May want to enhance with `prev` field, doubly linked list
  - Does not affect interface of `LinkedList`
- Instead, make `Node` a private class
  - Nested within `LinkedList`
  - Also called an **inner** class
- Objects of private class can see private components of enclosing class

```
public class LinkedList{
 private int size;
 private Node first;

 public Object head(){ ... }

 public void insert(Object newdata){
 ...
 }

 private class Node {
 public Object data;
 public Node next;
 ...
 }
}
```

# Manipulating objects

- Encapsulation is a key principle of object oriented programming
  - Internal data is private
  - Access to the data is regulated through public methods
  - Accessor and mutator methods
- Can ensure data integrity by regulating access

```
public class Date {
 private int day, month year;

 public void getDay(int d) {...}
 public void getMonth(int m) {...}
 public void getYear(int y) {...}

 public void setDay(int d) {...}
 public void setMonth(int m) {...}
 public void setYear(int y) {...}
}
```

# Manipulating objects

- Encapsulation is a key principle of object oriented programming
  - Internal data is private
  - Access to the data is regulated through public methods
  - Accessor and mutator methods
- Can ensure data integrity by regulating access
- Update date as a whole, rather than individual components
- Does this provide sufficient control?

```
public class Date {
 private int day, month year;

 public void getDay(int d) {...}
 public void getMonth(int m) {...}
 public void getYear(int y) {...}

 public void setDate(int d, int m, int y) {
 ...
 // Validate d-m-y combination
 }
}
```

# Querying a database

- Object stores train reservation information
  - Can query availability for a given train, date
- To control spamming by bots, require user to log in before querying
- Need to connect the query to the logged in status of the user
- “Interaction with state”

```
public class RailwayBooking {
 private BookingDB railwaydb;

 public int getStatus(int trainno, Date d) {
 // Return number of seats available
 // on train number trainno on date d
 ...
 }
}
```

# Querying a database

- Need to connect the query to the logged in status of the user
- Use objects!
  - On log in, user receives an object that can make a query
  - Object is created from private class that can look up `railwaydb`
- How does user know the capabilities of private class `QueryObject`?

```
public class RailwayBooking {
 private BookingDB railwaydb;

 public QueryObject login(String u, String p){
 QueryObject qobj;
 if (valid_login(u,p)) {
 qobj = new QueryObject();
 return(qobj);
 }
 }

 private class QueryObject {
 public int getStatus(int trainno, Date d) {
 // Return number of seats available
 // on train number trainno on date d
 ...
 }
 }
}
```

# Querying a database

- Need to connect the query to the logged in status of the user
- Use objects!
  - On log in, user receives an object that can make a query
  - Object is created from private class that can look up `railwaydb`
- How does user know the capabilities of private class `QueryObject`?
- Use an interface!
  - Interface describes the capability of the object returned on login

```
public interface QIF{
 public abstract int
 getStatus(int trainno, Date d);
}

public class RailwayBooking {
 private BookingDB railwaydb;
 public QIF login(String u, String p){
 QueryObject qobj;
 if (valid_login(u,p)) {
 qobj = new QueryObject();
 return(qobj);
 }
 }
 private class QueryObject implements QIF {
 public int getStatus(int trainno, Date d){
 ...
 }
 }
}
```

# Querying a database

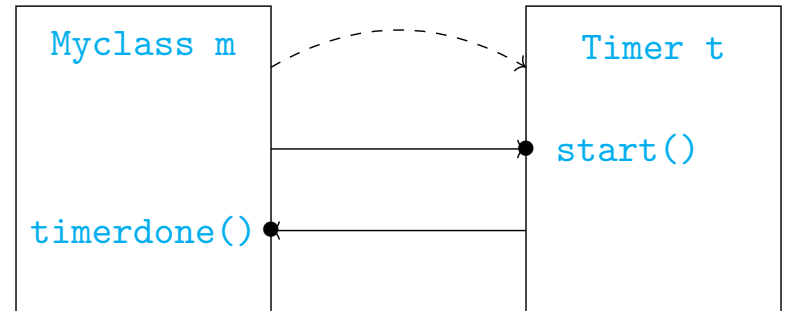
- Query object allows unlimited number of queries
- Limit the number of queries per login?
- Maintain a counter
  - Add instance variables to object returned on login
  - Query object can remember the **state** of the interaction

```
public class RailwayBooking {
 private BookingDB railwaydb;
 public QIF login(String u, String p){
 QueryObject qobj;
 if (valid_login(u,p)) {
 qobj = new QueryObject();
 return(qobj);
 }
 }
 private class QueryObject implements QIF {
 private int numqueries;
 private static int QLIM;

 public int getStatus(int trainno, Date d){
 if (numqueries < QLIM){
 // respond, increment numqueries
 }
 }
 }
}
```

# Implementing a call-back facility

- `Myclass m` creates a `Timer t`
- Start `t` to run in parallel
  - `Myclass m` continues to run
  - Will see later how to invoke parallel execution in Java!
- `Timer t` notifies `Myclass m` when the time limit expires
  - Assume `Myclass m` has a function `timerdone()`



## Implementing callbacks

- Code for `Myclass`
- `Timer t` should know whom to notify
  - `Myclass m` passes its identity when it creates `Timer t`
- Code for `Timer`
  - Interface `Runnable` indicates that `Timer` can run in parallel
- `Timer` specific to `Myclass`
- Create a generic `Timer`?

```
public class Myclass{

 public void f(){
 ..
 Timer t =
 new Timer(this);
 // this object
 // created t
 ...
 t.start(); // Start t
 ...
 }

 public void timerdone(){...}
}
```

```
public class Timer
 implements Runnable{
 // Timer can be
 // invoked in parallel

 private Myclass owner;

 public Timer(Myclass o){
 owner = o; // My creator
 }

 public void start(){
 ...
 owner.timerdone();
 // I'm done
 }
}
```

# A generic timer

- Use Java class hierarchy
- Parameter of `Timer` constructor of type `Object`
  - Compatible with all caller types
- Need to cast `owner` back to `Myclass`

```
public class Myclass{
 public void f(){
 ..
 Timer t =
 new Timer(this);
 // this object
 // created t
 ...
 t.start(); // Start t
 ...
 }

 public void timerdone(){...}
}
```

```
public class Timer
 implements Runnable{
 // Timer can be
 // invoked in parallel

 private Object owner;

 public Timer(Object o){
 owner = o; // My creator
 }

 public void start(){
 ...
 ((Myclass) owner).timerdone();
 // I'm done
 }
}
```

# Use interfaces

- Define an interface for callback

```
public interface
 Timerowner{

 public abstract
 void timerdone();
 }
```
- Modify `Myclass` to implement `Timerowner`
- Modify `Timer` so that `owner` is compatible with `Timerowner`

```
public class Myclass
 implements Timerowner{

 public void f(){
 ..
 Timer t =
 new Timer(this);
 // this object
 // created t
 ...
 t.start(); // Start t
 ...
 }

 public void timerdone(){...}
}
```

```
public class Timer
 implements Runnable{
 // Timer can be
 // invoked in parallel
 private Timerowner owner;

 public Timer(Timerowner o){
 owner = o; // My creator
 }

 public void start(){
 ...
 owner.timerdone();
 // I'm done
 }
}
```

# Linear list

- A generic linear list of objects
- Internal implementation may vary
- An array implementation

```
public class Linearlist {
 // Array implementation
 private int limit = 100;
 private Object[] data = new Object[limit];
 private int size; // Current size

 public Linearlist(){ size = 0; }

 public void append(Object o){
 data[size] = o;
 size++;
 ...
 }
 ...
}
```

# Linear list

- A generic linear list of objects
- Internal implementation may vary
- An array implementation
- A linked list implementation

```
public class Linearlist {
 private Node head;
 private int size;

 public Linearlist(){ size = 0; }

 public void append(Object o){
 Node m;

 for (m = head; m.next != null; m = m.next){}
 Node n = new Node(o);
 m.next = n;

 size++;
 }
 ...
 private class Node {...}
}
```

# Iteration

- Want a loop to run through all values in a linear list
- If the list is an array with public access, we write this
- For a linked list with public access, we could write this
- We don't have public access ...
- ...and we don't know which implementation is in use!

```
int i;
for (i = 0; i < data.length; i++){
 ... // do something with data[i]
}
```

```
Node m;
for (m = head; m != null; m = m.next){
 ... // do something with m.data
}
```

# Iterators

- Need the following abstraction

```
Start at the beginning of the list;
while (there is a next element){
 get the next element;
 do something with it
}
```

- Encapsulate this functionality in an interface called `Iterator`

```
public interface Iterator{
 public abstract boolean has_next();
 public abstract Object get_next();
}
```

# Iterators

- How do we implement `Iterator` in `Linearlist`?
- Need a “pointer” to remember position of the iterator
- How do we handle nested loops?

```
for (i = 0; i < data.length; i++){
 for (j = 0; j < data.length; j++){
 ... // do something with data[i] and data[j]
 }
}
```

# Iterators

- Solution: Create an `Iterator` object and export it!

```
public class Linearlist{

 private class Iter implements Iterator{
 private Node position;
 public Iter(){...} // Constructor
 public boolean has_next(){...}
 public Object get_next(){...}
 }

 // Export a fresh iterator
 public Iterator get_iterator(){
 Iter it = new Iter();
 return(it);
 }
}
```

- Definition of `Iter` depends on linear list

- Now, we can traverse the list externally as follows:

```
Linearlist l = new Linearlist();
...
Object o;
Iterator i = l.get_iterator();

while (i.has_next()){
 o = i.get_next();
 ... // do something with o
}
...
```

- For nested loops, acquire multiple iterators!

```
Linearlist l = new Linearlist();
...
Object oi,oj;
Iterator i,j;

i = l.get_iterator();
while (i.has_next()){
 oi = i.get_next();
 j = l.get_iterator();
 while (j.has_next()){
 oj = j.get_next();
 ... // do something with oi, oj
 }
}
...
```